

INDIVIDUAL LAB - WORK RECORD

MODULE 2

Unit Test Frameworks

Assignment 7: Data-Driven Login Automation Using External Test Data

Student Name: Protyusha Mondal

Roll No.: 12023052002072

Assignments: 7, 8 and 9

Field	Details
Module Name	Unit Test Frameworks
Assignment Title	Data-Driven Login Automation Using External Test Data
Framework	Python, PyTest and Selenium WebDriver
Test Data Source	External CSV file
Target URL	https://the-internet.herokuapp.com/login

1. Problem Statement

Develop a data-driven login automation test that reads multiple username and password combinations from an external CSV file and verifies the expected validation result for each test case.

2. Objective

- Read test data from an external CSV file.
- Execute multiple login scenarios.
- Test valid and invalid credentials.
- Verify the expected validation message using assertions.
- Use PyTest parameterization to execute multiple test cases.

3. Tools, Software and Concepts Used

- Python
- PyTest
- Selenium WebDriver
- CSV
- PyTest parameterization
- Assertions
- Chrome WebDriver

4. Implementation

4.1 Project Structure

```
Experiment_7_Data_Driven_Testing/  
|  
├── login_data.csv  
└── test_login_ddt.py
```

4.2 External Test Data — login_data.csv

```
username,password,expected
tomsmith,SuperSecretPassword!,valid
invaliduser,wrongpass,invalid
admin,wrongpass,invalid
```

4.3 PyTest Selenium Test — test_login_ddt.py

```
import csv
import pytest
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

def read_login_data():
    with open("login_data.csv", newline="") as file:
        return list(csv.DictReader(file))

@pytest.mark.parametrize("data", read_login_data())
def test_login(data):
    driver = webdriver.Chrome()
    try:
        # Selenium opens the login page
        driver.get("https://the-internet.herokuapp.com/login")
        wait = WebDriverWait(driver, 10)

        # Enter username
        username = wait.until(
            EC.presence_of_element_located((By.ID, "username"))
        )
        username.send_keys(data["username"])

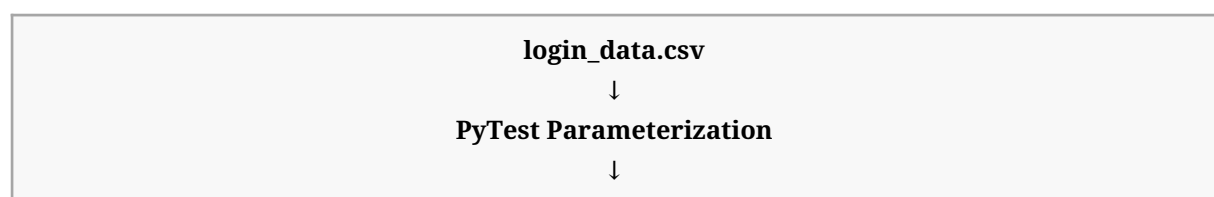
        # Enter password
        password = driver.find_element(By.ID, "password")
        password.send_keys(data["password"])

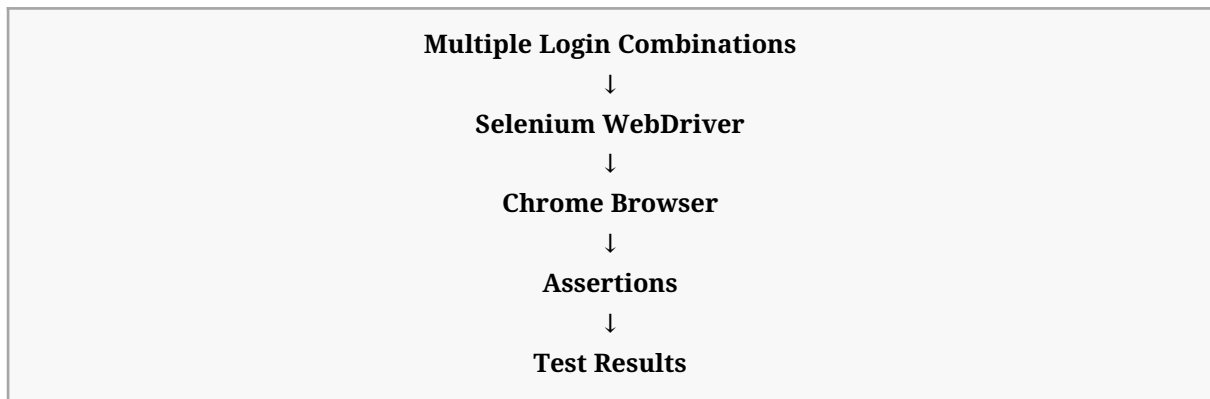
        # Click Login
        driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()

        # Read validation/result message
        message = wait.until(
            EC.visibility_of_element_located((By.ID, "flash"))
        ).text

        # Verify expected result
        if data["expected"] == "valid":
            assert "You logged into a secure area!" in message
        else:
            assert "Your username is invalid!" in message or \
                "Your password is invalid!" in message
    finally:
        driver.quit()
```

4.4 Data-Driven Test Flow





4.5 Expected Output

```
rootdir: C:\project_3rd_year\Selenium_Assignment\Module_2_Unit_Test_Frameworks\Experiment_7_Data_Driven_Testing
plugins: anyio-4.9.0, html-4.2.0, metadata-3.1.1
collected 3 items

test_login_ddt.py::test_login[data0] PASSED [ 33%]
test_login_ddt.py::test_login[data1] PASSED [ 66%]
test_login_ddt.py::test_login[data2] PASSED [100%]

===== 3 passed in 26.23s =====
```

Figure 7: Data-driven Selenium login automation using pytest parameterization and external CSV test data

5. Result / Observation

The login test successfully read multiple username and password combinations from an external CSV file. pytest parameterization executed the login test separately for each data set. Selenium automated the browser interaction, and assertions verified the expected validation result for both valid and invalid credentials.

6. Conclusion

Data-driven testing was successfully implemented using an external CSV file, pytest parameterization, Selenium WebDriver, and assertions. Multiple login scenarios were executed automatically using the same test function.

Assignment 8: PyTest Integration with HTML Reporting and Failure Screenshots

Field	Details
Module Name	Unit Test Frameworks
Assignment Title	Integrating PyTest with HTML Reporting and Automatic Screenshot Capture on Test Failure
Framework	Python, PyTest and Selenium WebDriver
Reporting	PyTest-HTML
Browser	Chrome WebDriver

1. Problem Statement

Execute a Selenium-based test using the PyTest framework and generate an HTML execution report. The test framework should use fixtures for browser initialization and teardown and automatically capture a screenshot whenever a Selenium test fails.

2. Objective

- Execute Selenium test cases using PyTest.
- Use PyTest fixtures for browser setup and teardown.
- Generate an HTML execution report using the PyTest-HTML plugin.
- Detect failed Selenium test cases using a PyTest hook.
- Automatically capture and save a screenshot when a test fails.
- Verify the test execution results through the generated HTML report.

3. Tools, Software and Concepts Used

- Python
- PyTest
- Selenium WebDriver
- PyTest-HTML
- Chrome WebDriver
- PyTest Fixtures
- PyTest Hooks
- HTML Reporting
- Selenium Screenshots
- Assertions

4. Implementation

4.1 Project Structure

```
Experiment_8_PyTest_HTML_Reporting/  
├──  
├── screenshots/  
├── reports/  
├── conftest.py  
└── test_failure.py
```

4.2 Source Code / Implementation — conftest.py

```
import os
import pytest
from selenium import webdriver

@pytest.fixture
def driver():
    driver = webdriver.Chrome()
    driver.maximize_window()
    yield driver
    driver.quit()

@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    report = outcome.get_result()

    if report.when == "call" and report.failed:
        driver = item.funcargs.get("driver")
        if driver:
            os.makedirs("screenshots", exist_ok=True)
            screenshot_path = os.path.join(
                "screenshots",
                f"{item.name}.png"
            )
            driver.save_screenshot(screenshot_path)
            print(f"\nScreenshot saved: {screenshot_path}")
```

The driver fixture initializes the Chrome browser before each test and closes it after the test using `driver.quit()`.

The `pytest_runtest_makereport` hook checks whether a test has failed. If the test fails during the call phase, the Selenium WebDriver instance is retrieved and a screenshot is captured and saved in the screenshots directory.

4.3 Source Code / Implementation — test_failure.py

```
def test_successful_page(driver):
    driver.get("https://www.saucedemo.com/")
    title = driver.title
    assert title == "Swag Labs"

def test_failed_page(driver):
    driver.get("https://www.saucedemo.com/")
    title = driver.title
    # Intentional failure to demonstrate screenshot capture
    assert title == "Wrong Title"
```

The second test contains an intentional incorrect assertion so that the automatic screenshot functionality can be demonstrated.

4.4 HTML Report Generation

The PyTest HTML plugin can be used to generate a self-contained HTML report.

Execute the test using:

```
python -m pytest test_failure.py -v --html=reports/report.html --self-contained-html
```

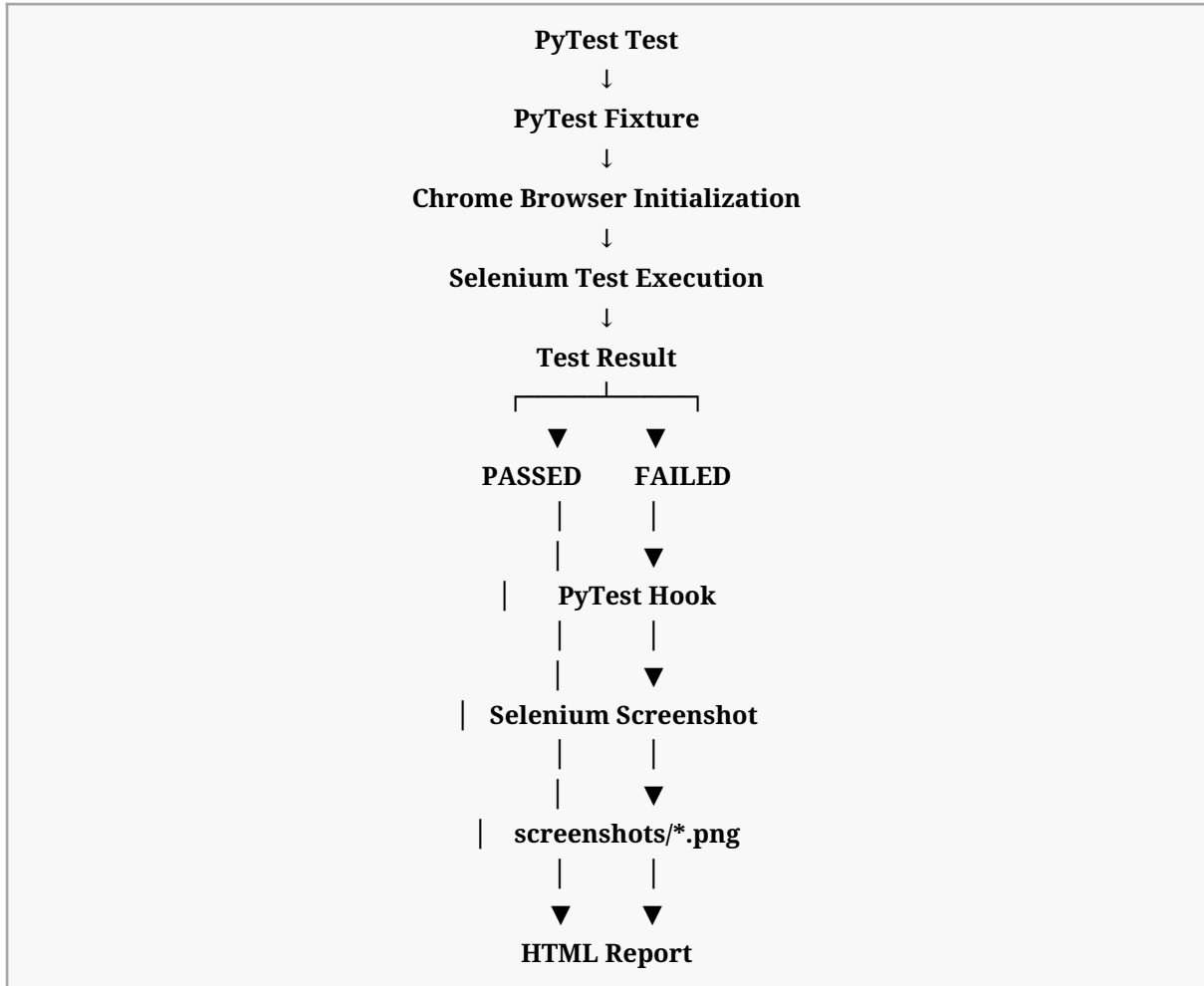
The generated report is stored at:

```
reports/report.html
```

The screenshot generated after a failed test is stored at:

screenshots/test_failed_page.png

4.5 Execution Flow



4.6 Output

```
rootdir: C:\project_3rd_year\Selenium_Assignment\Module_2_Unit_Test_Frameworks\Experiment_11_Screenshot_Failure
plugins: anyio-4.9.0, html-4.2.0, metadata-3.1.1
collected 2 items

test_failure.py::test_successful_page PASSED [ 50%]
test_failure.py::test_failed_page
Screenshot saved: screenshots\test_failed_page.png
FAILED [100%]

===== FAILURES =====
----- test_failed_page -----

driver = <selenium.webdriver.chrome.webdriver.WebDriver (session="afe39b3425e301ebb9877baa6653591c")>

    def test_failed_page(driver):
        driver.get("https://www.saucedemo.com/")

        title = driver.title

        # Intentional failure to demonstrate screenshot capture
> assert title == "Wrong Title"
E       AssertionError: assert 'Swag Labs' == 'Wrong Title'
E
E       - Wrong Title
E       + Swag Labs
E

test_failure.py:18: AssertionError
===== short test summary info =====
FAILED test_failure.py::test_failed_page - AssertionError: assert 'Swag Labs' == 'Wrong Title'
===== 1 failed, 1 passed in 9.33s =====
```

Figure 8: PyTest failure output with automatic screenshot capture

```
rootdir: C:\project_3rd_year\Selenium_Assignment\Module_2_Unit_Test_Frameworks\Experiment_8_HTML_Reporting
plugins: anyio-4.9.0, html-4.2.0, metadata-3.1.1
collected 1 item

test_report.py::test_ecommerce_page PASSED [100%]

- Generated html report: file:///C:/project_3rd_year/Selenium_Assignment/Module_2_Unit_Test_Frameworks/Experiment_8_HTML_Reporting/reports/report.html -
===== 1 passed in 4.42s =====
```

Figure 9: PyTest HTML execution report

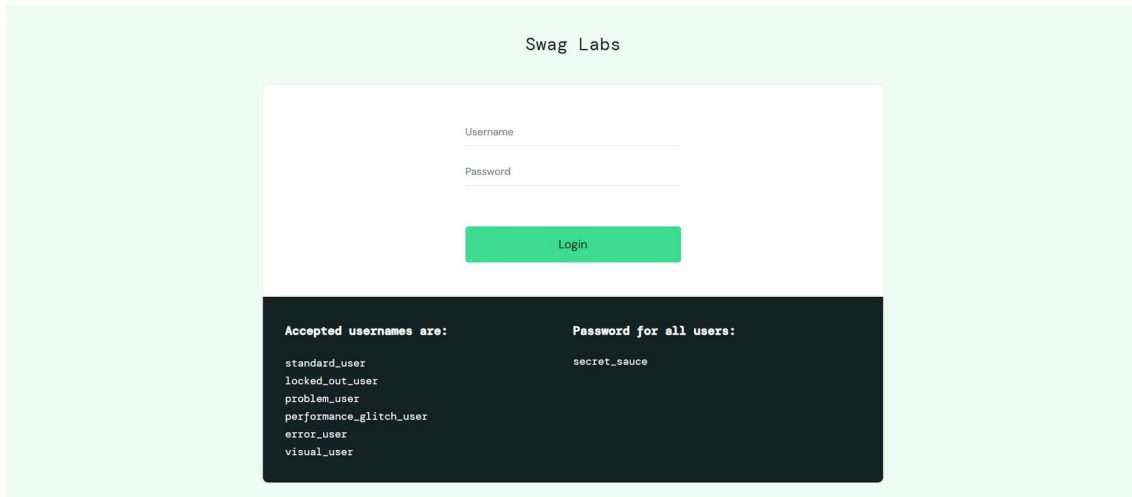


Figure 10: Screenshot automatically captured after Selenium test failure

The generated HTML report displays the test execution results.

5. Result / Observation

The Selenium test cases were successfully executed using PyTest. The WebDriver was initialized and terminated using a PyTest fixture. The PyTest-HTML plugin successfully generated an HTML execution report containing the test results.

When the intentionally failed test was executed, the PyTest hook detected the failure and automatically captured a screenshot of the Selenium browser. The screenshot was saved in the designated screenshots directory.

6. Conclusion

PyTest was successfully integrated with Selenium WebDriver for automated browser testing. Fixtures were used to manage browser initialization and teardown, while the PyTest-HTML plugin was used to generate an HTML execution report. A PyTest hook was also implemented to automatically capture screenshots when Selenium test cases failed, providing useful information for debugging failed test executions.

Assignment 9: Page Object Model

Field	Details
Module Name	Unit Test Frameworks
Assignment Title	Implementing Page Object Model for Selenium Automation
Design Pattern	Page Object Model (POM)
Framework	Python, Selenium WebDriver and PyTest
Target URL	https://the-internet.herokuapp.com/login

1. Problem Statement

Implement the Page Object Model (POM) design pattern for Selenium automation by separating page locators and UI interaction methods from test cases.

2. Objective

- Understand the Page Object Model design pattern.
- Create a reusable BasePage class.
- Create a LoginPage class containing page locators and UI methods.
- Separate UI interaction from test assertions.
- Execute Selenium tests using PyTest.

3. Tools, Software and Concepts Used

- Python
- Selenium WebDriver
- PyTest
- Page Object Model
- Locators
- Base Page
- Page-specific methods
- Assertions

4. Implementation

4.1 Project Structure

```
project/
├── pages/
│   ├── __init__.py
│   ├── base_page.py
│   └── login_page.py
├── tests/
│   └── test_login.py
└── conftest.py
```

4.2 Source Code / Implementation — pages/base_page.py

```
from selenium.webdriver.support.ui import WebDriverWait

class BasePage:
    def __init__(self, driver):
        self.driver = driver
        self.wait = WebDriverWait(driver, 10)

    def open(self, url):
        self.driver.get(url)

    def get_title(self):
        return self.driver.title
```

4.3 Source Code / Implementation — pages/login_page.py

```
from selenium.webdriver.common.by import By
from pages.base_page import BasePage

class LoginPage(BasePage):
    # Locators
    USERNAME = (By.ID, "username")
    PASSWORD = (By.ID, "password")
    LOGIN_BUTTON = (By.CSS_SELECTOR, "button[type='submit']")
    MESSAGE = (By.ID, "flash")

    def enter_username(self, username):
        self.driver.find_element(*self.USERNAME).send_keys(username)

    def enter_password(self, password):
        self.driver.find_element(*self.PASSWORD).send_keys(password)

    def click_login(self):
        self.driver.find_element(*self.LOGIN_BUTTON).click()

    def get_message(self):
        return self.wait.until(
            lambda driver: driver.find_element(*self.MESSAGE).text
        )

    def login(self, username, password):
        self.enter_username(username)
        self.enter_password(password)
        self.click_login()
```

4.4 Source Code / Implementation — conftest.py

```
import pytest
from selenium import webdriver

@pytest.fixture
def driver():
    driver = webdriver.Chrome()
    driver.maximize_window()
    yield driver
    driver.quit()
```

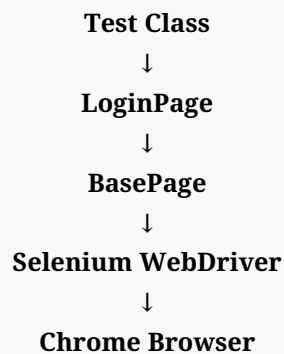
4.5 Source Code / Implementation — tests/test_login.py

```
from pages.login_page import LoginPage

def test_valid_login(driver):
    login_page = LoginPage(driver)
    login_page.open("https://the-internet.herokuapp.com/login")
    login_page.login("tomsmith", "SuperSecretPassword!")
    message = login_page.get_message()

    # Assertion remains in the test class/function
    assert "You logged into a secure area!" in message
```

4.6 POM Architecture



- BasePage — provides common browser/page functionality.
- LoginPage — contains login locators and UI interaction methods.
- Test class/function — performs the test and contains assertions.
- conftest.py — provides the Selenium WebDriver fixture.

4.7 Execution

From the project directory, execute:

```
pytest tests/test_login.py -v
```

4.8 Output

```
rootdir: C:\project_3rd_year\Selenium_Assignment\Module_2_Unit_Test_Frameworks\Experiment_9_Page_Object_Model
plugins: anyio-4.9.0, html-4.2.0, metadata-3.1.1
collected 1 item

tests/test_login.py::test_valid_login PASSED [100%]

===== 1 passed in 8.88s =====
```

Figure 11: Selenium login automation implemented using the Page Object Model

5. Result / Observation

The Page Object Model was successfully implemented by separating common browser operations, page-specific locators and UI methods, and test assertions into different components. The LoginPage class handled the login interaction while the test function verified the expected result using an assertion.

6. Conclusion

The Page Object Model design pattern was successfully implemented for Selenium automation. The separation of page objects from test logic improved the organization, readability, and reusability of the Selenium test code.